

CSC1031: OOP

Misha

Java ecosystem: JDK, JVM, JRE

JDK (Java Development Kit)

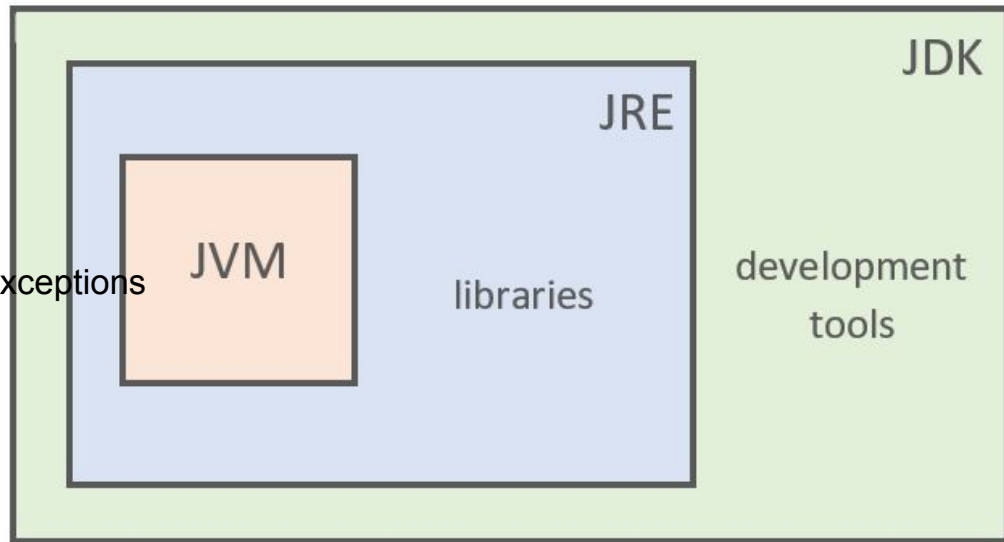
- For **developers**
- Includes compiler (`javac`), debugger, tools, libraries
- Converts `.java` → `.class` (bytecode)

JVM (Java Virtual Machine)

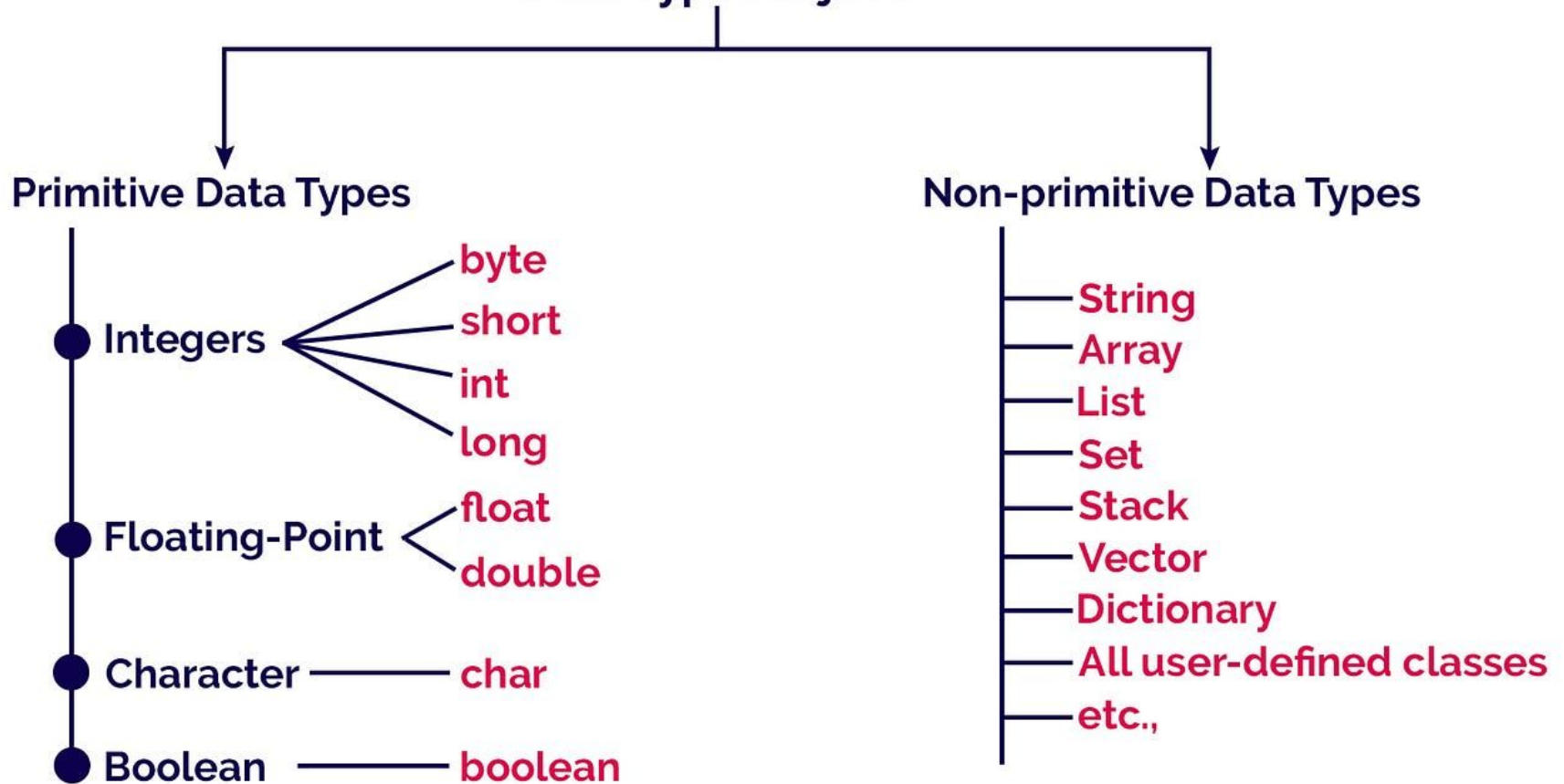
- For **execution**
- Interprets bytecode (`.class`)
- Handles memory, garbage collection, runtime exceptions

JRE (Java Runtime Environment)

- For **running apps**
- Contains JVM + core libraries
- Needed by end-users, not for development



Data Types in java



Reference Types

```
int number;
```

```
number = 237;
```

```
number = 35;
```

number

237

```
Customer customer;
```

```
customer = new Customer();
```

```
customer = new Customer();
```

customer



Data Structures: Arrays, Lists, Maps.

Arrays

- Fixed size, indexed
- Fast access: `arr[0]`
- Declaration: `int[] arr = {1, 2, 3};`

ArrayList (part of `java.util`)

- Resizable, dynamic
- Only stores objects

```
ArrayList<String> names = new ArrayList<>();  
names.add("Alice");
```

Key Difference:

- `Array` = fixed size
- `ArrayList` = flexible, grows as needed

Data Structures: Arrays, Lists, Maps.

Map<K, V>

- Stores pairs: **key** → **value**
- Fast lookup via key
- Think of a dictionary

```
Map<String, Integer> scores = new HashMap<>();  
scores.put("Alice", 90);  
int aliceScore = scores.get("Alice");
```

Shallow Copy – Copying the Reference

Shallow Copy creates a new reference to the same object in memory.

```
Person p1 = new Person("Alice");  
Person p2 = p1; // shallow copy  
p2.name = "Bob";  
System.out.println(p1.name); // Output: Bob
```

Both **p1** and **p2** refer to the **same object** → changes affect both.

Risk: Accidental side effects when modifying one affects the other.

Deep Copy – Full Duplication

Deep Copy creates a completely new object with the same data.

No shared references — changes to the copy do not affect the original.

```
class Person {  
    String name;  
    Person(String name) { this.name = name; }  
  
    Person copy() {  
        return new Person(this.name); // deep copy  
    }  
}
```


OOP Concepts: Class, Object, Attributes, Methods

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around objects — real-world entities with state and behavior.

Class = Blueprint

- A class defines the structure and behavior of objects.
- Example: `class Dog { String name; void bark(); }`

Object = Instance

- A specific realization of a class.
- Example: `Dog myDog = new Dog();`

OOP Concepts: Class, Object, Attributes, Methods

Attributes (Fields)

- Represent the state of an object.
- Declared as variables inside the class.
- Example: `String name; int age;`

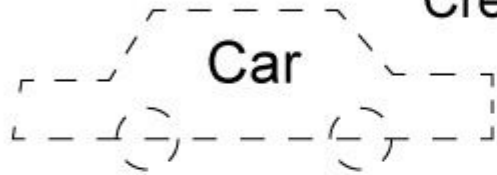
Methods

- Define what the object can do (behavior).
- Example: `void bark() { System.out.println("Woof!"); }`

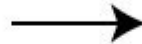
Objects interact by calling methods on one another and modifying their attributes.

OOP Concepts: Class, Object, Attributes, Methods

Class



Create an instance



Object



Properties

color

price

km

model

Methods - behaviors

start()

backward()

forward()

stop()

Property values

color: red

price: 23,000

km: 1,200

model: Audi

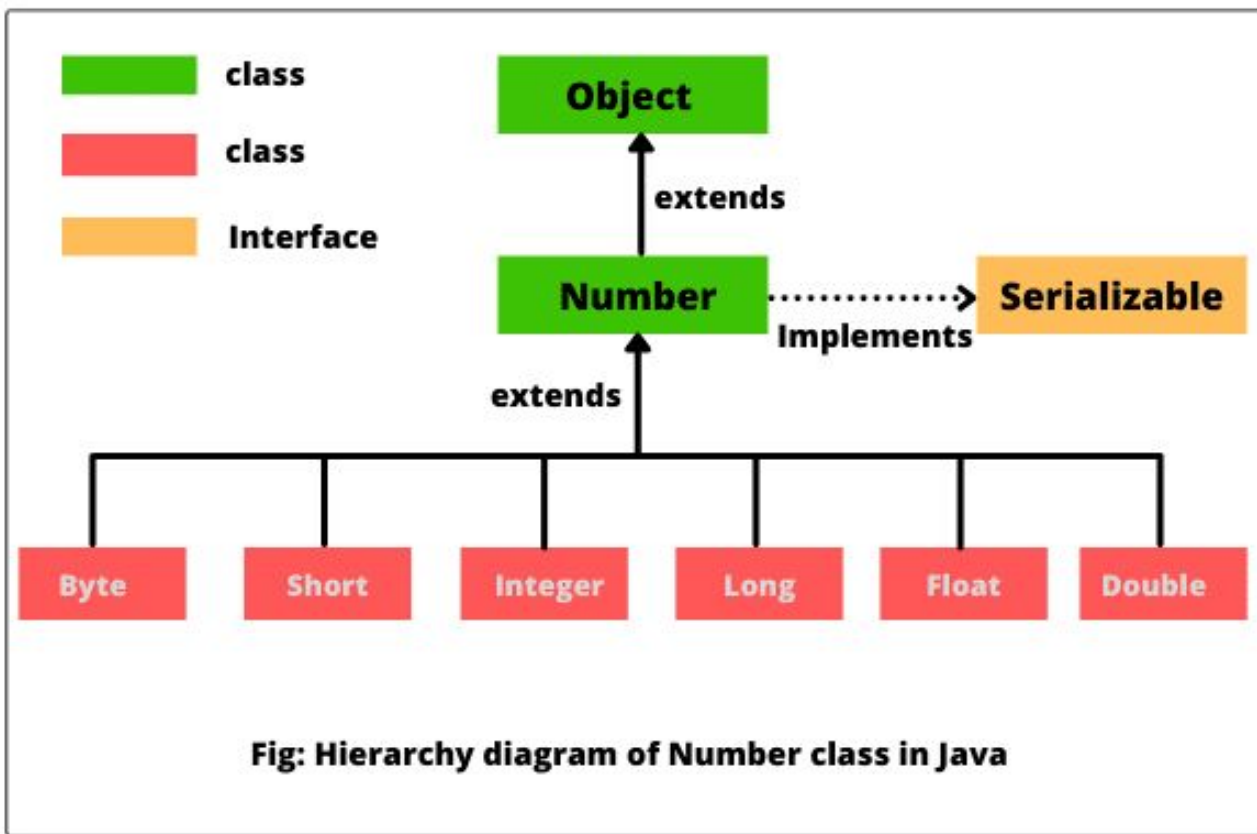
Methods

start()

backward()

forward()

stop()



Encapsulation – Hiding the Internals

Encapsulation means wrapping data and behavior together and hiding internal details from the outside.

- Protects internal state
- Provides controlled access via methods
- Increases security and maintainability

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) { ... }  
    public double getBalance() { return balance; }  
}
```

Access Modifiers

Modifier	Class	Package	SubClass	Global
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
Default	✓	✓	✗	✗
private	✓	✗	✗	✗

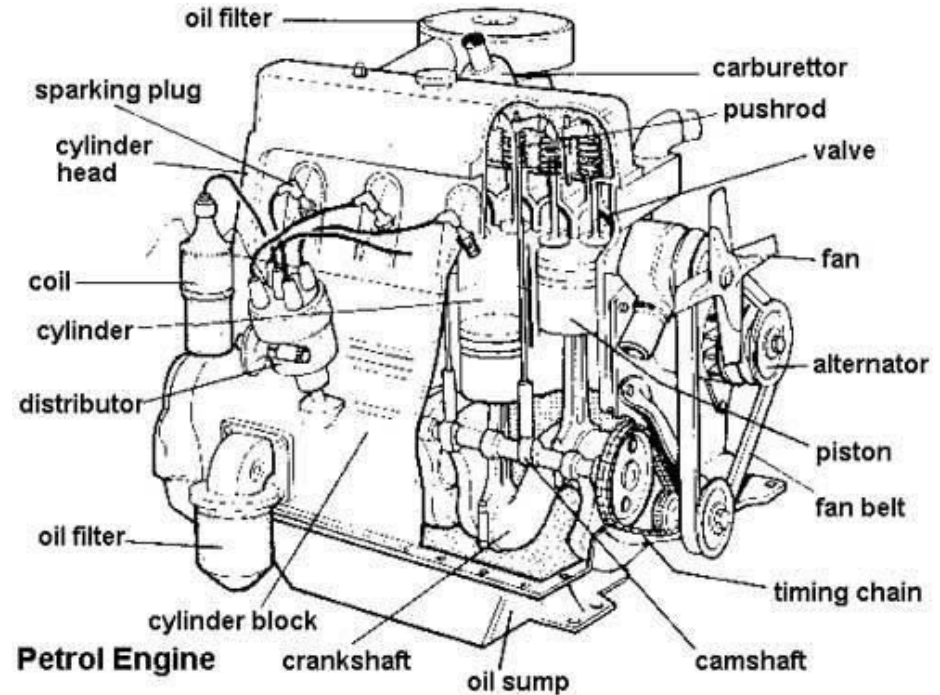
Abstraction – Focus on What, Not How

Abstraction means showing only the essential features and hiding the complex implementation.

It answers: “What can this object do?”
— not “How does it do it?”

Real-world example:

You drive a car without needing to understand the engine.

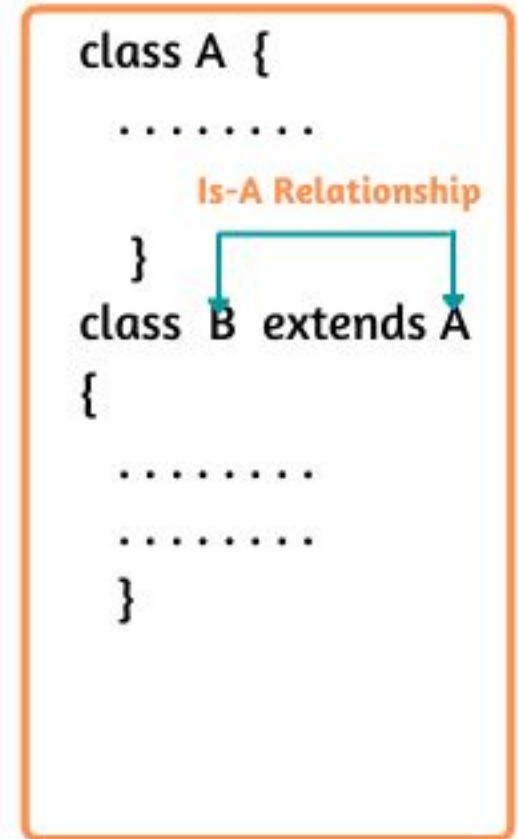
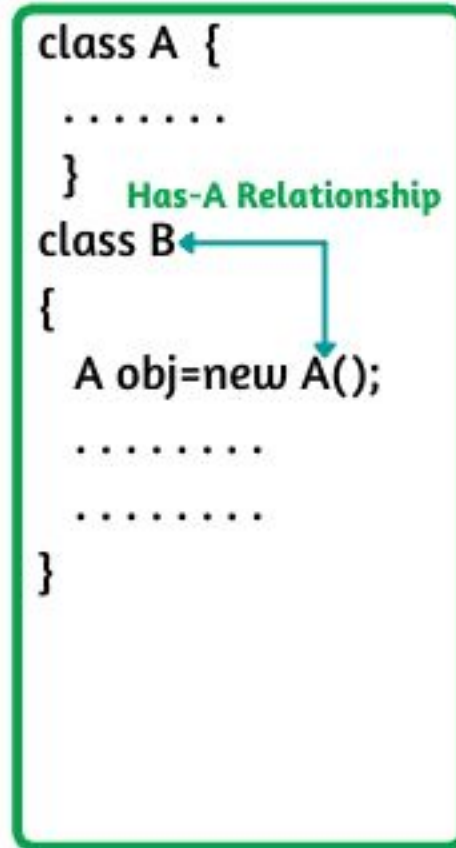
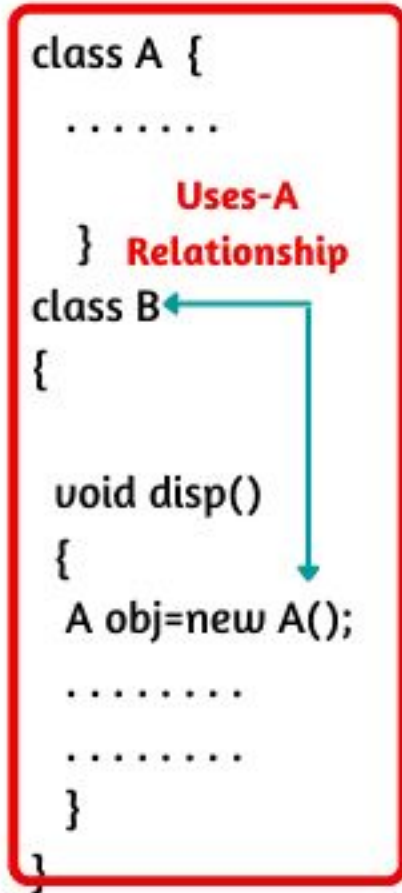


Object Relationships Overview

In OOP, classes often **connect to each other** — these are called *relationships*:

1. **Association** – General connection (uses or knows about)
→ *"Student attends Course"*
2. **Aggregation** – Weak “has-a”
→ *"Department has Professors"*, but professors can exist elsewhere
3. **Composition** – Strong “has-a”
→ *"House has Rooms"*, if house is deleted → rooms go too

Object Relationships Overview



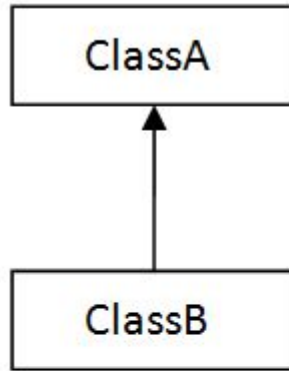
Inheritance: Single, Multilevel, Hierarchical

Inheritance = mechanism where a child class inherits fields and methods from a parent class.

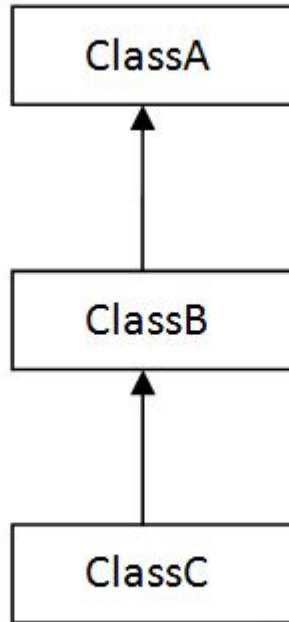
- Code reuse
- Extensibility
- Clear structure

```
class Animal {  
    void eat() { System.out.println("Eating..."); }  
}  
  
class Dog extends Animal {  
    void bark() { System.out.println("Barking..."); }  
}
```

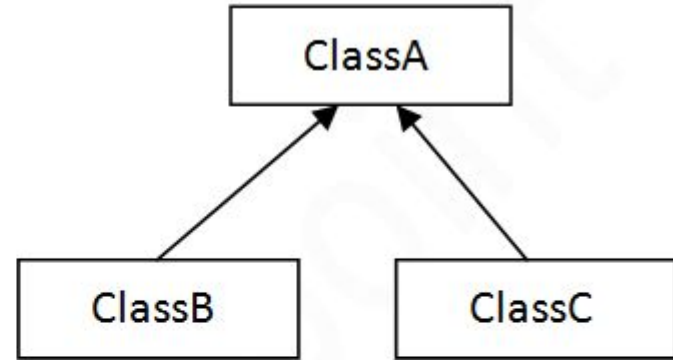
Inheritance: Single, Multilevel, Hierarchical



1) Single



2) Multilevel



3) Hierarchical

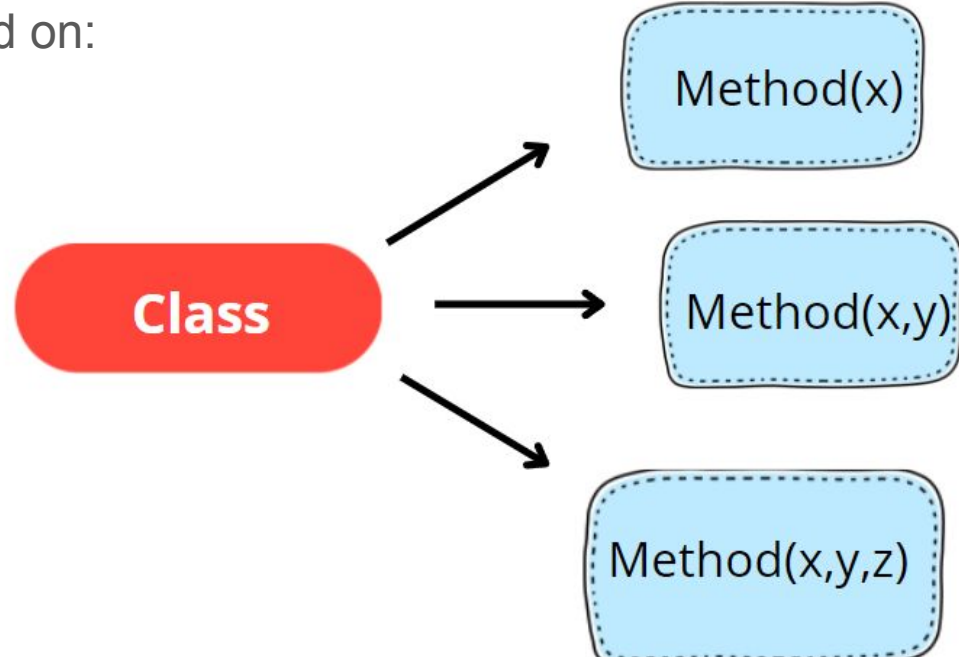
Method Overloading NOT overriding

Method Overloading = multiple methods with the same name, but different parameters in the same class.

Java decides which method to run based on:

- Number of parameters
- Type of parameters
- Order of parameters

```
void print() {}  
void print(String text) {}  
void print(int x, double y) {}
```



What is Method Overriding?

Method Overriding = when a subclass provides a new implementation of a method inherited from its superclass.

Must match:

- Method name
- Parameters
- Return type

You cannot override:

- final methods
- private methods
- static methods

```
class Animal {  
    void speak() {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void speak() {  
        System.out.println("Woof!");  
    }  
}
```

Constructor Chaining

Constructor Chaining = calling one constructor from another within the same class using this(...)

- Avoid duplication
- Centralize initialization logic
- Simplify code structure

```
class Rectangle {  
    int width, height;  
  
    Rectangle() {  
        this(1, 1); // Calls the constructor below  
    }  
  
    Rectangle(int w, int h) {  
        width = w;  
        height = h;  
    }  
}
```

Static: variables, methods, blocks, inner classes.

static keyword means "belongs to the class, not to instances"

Static Variable

- Shared across all instances

Static Method

- Can be called without creating an object
- Cannot access non-static members directly

Static: variables, methods, blocks, inner classes.

Static Block

- Runs once when the class is loaded

Static Inner Class

- Can be created without an outer class instance
- Cannot access non-static members of the outer class
- Use when nested logic doesn't need outer class state.

Interfaces – Behavior Contracts

An interface defines what a class must do, not how it does it.

- A class can implement multiple interfaces
- Methods are implicitly public and abstract
- Can include default and static methods

```
interface Flyable {  
    void fly();  
}
```

Use when:

- You want to define a capability (e.g., Serializable, Comparable)
- You need multiple inheritance of type

Default Methods

Default methods are methods in an interface that have a concrete implementation.

```
interface Greeter {  
    default void greet() {  
        System.out.println("Hello!");  
    }  
}
```

Classes can inherit or override this method

Interfaces can now contain behavior, not just contracts

Abstract Classes – Partial Implementation

An abstract class is a base class with shared code + abstract methods.

- Can have fields, constructors, and non-abstract methods
- Cannot be instantiated
- A class can only extend one abstract class

```
abstract class Animal {  
    abstract void speak();  
    void sleep() { System.out.println("Zzz"); }  
}
```

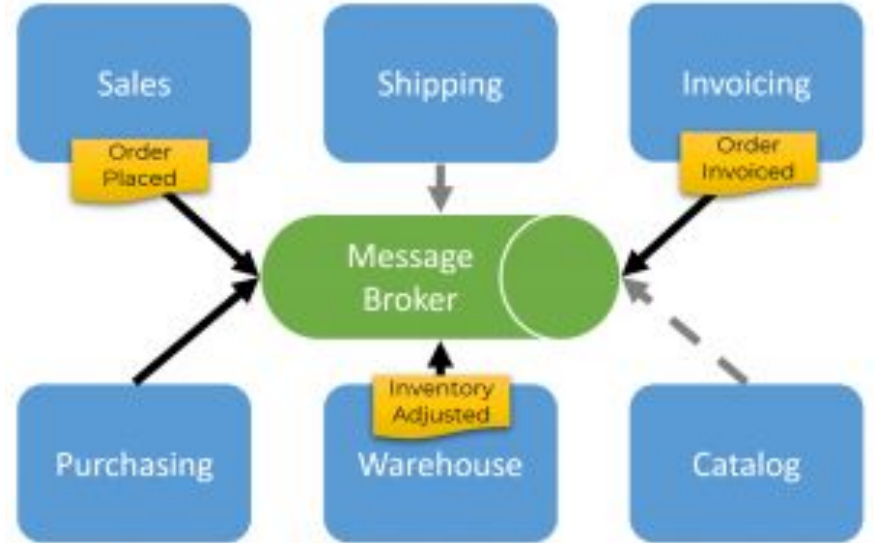
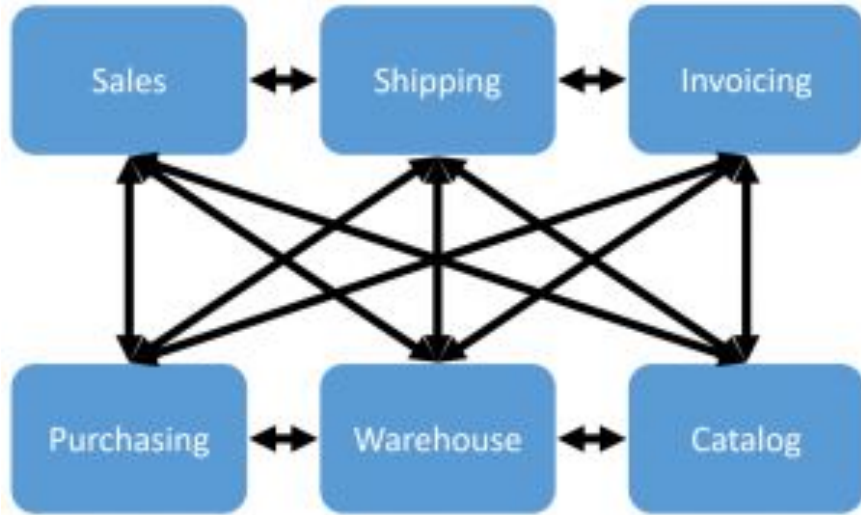
Use when:

- You need shared state or logic across related classes
- You're modeling a general concept with common base behavior

Tight Coupling

vs

Loose Coupling



What is Loose Coupling?

Loose coupling means that components (classes, modules) are minimally dependent on each other.

Each class should know as little as possible about other classes it uses.

Benefits:

- Easier to test
- Easier to maintain
- Easier to reuse and extend

Tight coupling example:

```
class OrderService {  
    PaymentService payment = new PaymentService(); // tightly coupled  
}
```

How to Achieve Loose Coupling

Use Interfaces

Use design patterns like:

- Strategy
- Observer
- Factory
- Decorator

Key Idea: Depend on abstractions, not concrete implementations.

```
class OrderService {  
    PaymentProcessor processor;  
  
    OrderService(PaymentProcessor processor) {  
        this.processor = processor;  
    }  
}
```

UML

UML symbols

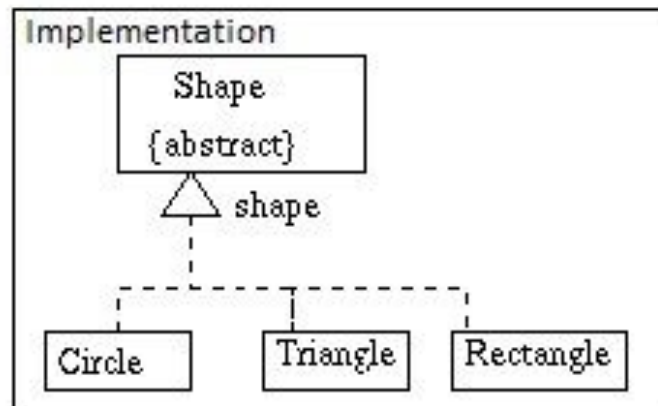
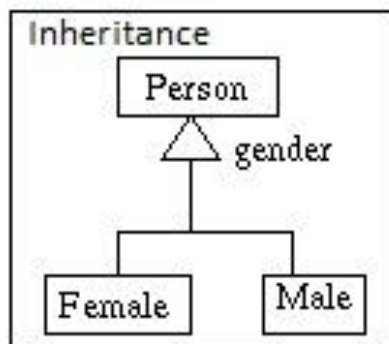
Association	Symbol
Composition	
Aggregation	
Inheritance	
Implementation	



Composition: every car has an engine.



Aggregation: cars may have passengers, they come a



Generics: type-safe programming

Generics allow you to write type-safe and reusable code without specifying exact data types upfront.

```
class Box<T> {  
    T item;  
    void set(T item) { this.item = item; }  
    T get() { return item; }  
}
```

```
public <T> void print(T value) {  
    System.out.println(value);  
}
```


Design patterns

Design Patterns are **proven solutions** to common software design problems.

Patterns are not code, but design blueprints that guide how you build systems.

Categories:

- Creational (e.g., Singleton, Factory)
- Structural (e.g., Decorator, Adapter)
- Behavioral (e.g., Strategy, Observer)

Why?

Standardized vocabulary

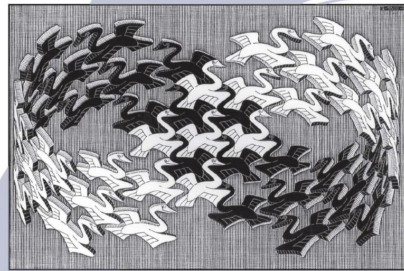
Encourage best practices

Solve recurring problems

Design Patterns

Elements of Reusable Object-Oriented Software

Erich Gamma
Richard Helm
Ralph Johnson
John Vlissides



Cover art © 1994 M.C. Escher / Cordon Art - Baarn - Holland. All rights reserved.

Foreword by Grady Booch